

Phase 1: JavaScript Mastery - Module 4: Functions and Execution Context

Let's start module 4: Functions and Execution Context and the first topic is What is a Function

Module 4: Functions and Execution Context

Topic 1: What is a Function?

A **function** is a reusable block of code that performs a specific task.

Instead of writing the same code multiple times, we place it inside a function and call it whenever needed.

Real-Life Analogy

Think of a function like a **coffee machine**.

- You press a button (call the function).
- The machine performs some work.
- It gives you coffee (returns a result).

You don't need to know every internal detail each time; you simply use it whenever required.

Why Do We Need Functions?

Without functions:

```
JavaScript
console.log("Welcome Krishna");
console.log("Welcome Krishna");
console.log("Welcome Krishna");
```

With functions:

```
JavaScript
function greet() {
  console.log("Welcome Krishna");
}

greet();
greet();
greet();
```

Benefits

- Code Reusability
- Better Organization
- Easier Maintenance
- Reduces Repetition
- Makes Programs Modular

Function Syntax

```
JavaScript
function functionName() {
    // code to execute
}
```

Example:

```
JavaScript
function sayHello() {
    console.log("Hello World");
}
```

Calling the function:

```
JavaScript
sayHello();
```

Output:

```
JavaScript
Hello World
```

Parts of a Function

```
JavaScript
function add(a, b) {
    return a + b;
}
```

Breakdown

1. **function**

- Keyword used to create a function.

2. **add**

- Function name.

3. **a, b**

- Parameters (inputs).

4. return

- Sends a value back.

5. a + b

- Function body logic.

Function with Parameters

Parameters allow a function to receive data.

```
JavaScript
function greet(name) {
  console.log("Hello " + name);
}

greet("Krishna");
greet("Rahul");
```

Output:

```
JavaScript
Hello Krishna
Hello Rahul
```

Here:

- name → parameter
- "Krishna" → argument

Parameter vs Argument

```
JavaScript
function greet(name) {
  console.log(name);
}

greet("Krishna");
```

Term	Meaning
Parameter	Variable inside function definition
Argument	Actual value passed during function call

Function with Return Value

```
JavaScript
function add(a, b) {
  return a + b;
}

let result = add(10, 20);
```

```
console.log(result);
```

Output:

```
JavaScript
30
```

The **return** keyword:

- Ends function execution.
- Sends a value back to the caller.

Function Without Return

```
JavaScript
function add(a, b) {
  console.log(a + b);
}

let result = add(10, 20);

console.log(result);
```

Output:

```
JavaScript
30
undefined
```

Why?

Because the function doesn't return anything.

JavaScript automatically returns:

```
JavaScript
undefined
```

Function Execution Flow

```
JavaScript
function greet() {
  console.log("Hello");
}

console.log("Start");

greet();

console.log("End");
```

Execution Order:

```
Start
Hello
End
```

JavaScript executes code line by line.

Function Declaration Example

```
JavaScript
function square(num) {
  return num * num;
}

console.log(square(5));
```

Output:

```
JavaScript
25
```

Memory View (Basic)

```
JavaScript
function add(a, b) {
  return a + b;
}

let result = add(5, 10);
```

Global Memory

```
add → function definition
result → 15
```

During Function Call

```
Function Memory

a → 5
b → 10
```

After execution:

```
result → 15
```

Function memory is created temporarily and destroyed after execution.

Key Points for Interviews

Q1. What is a function?

A function is a reusable block of code designed to perform a particular task.

Q2. Why use functions?

- Reusability
 - Modularity
 - Maintainability
 - Less code duplication
-

Q3. What is the difference between parameters and arguments?

Parameters are variables declared in the function definition, while arguments are actual values passed during the function call.

Q4. What happens if a function doesn't return anything?

JavaScript automatically returns `undefined`.

Module 4 Connection

Functions are the foundation of JavaScript.

The next topics:

1. Function Declaration
2. Function Expression
3. Arrow Functions
4. Scope
5. Execution Context
6. Call Stack
7. Hoisting of Functions

Once we reach **Execution Context**, you'll understand what actually happens inside the JavaScript engine when a function is called. That's one of the most important interview topics in JavaScript.

Topic 2: Function Declaration, Expression, Arrow Functions

Topic 2: Function Declaration, Function Expression, and Arrow Functions

JavaScript provides multiple ways to create functions. The three most common are:

1. Function Declaration
2. Function Expression
3. Arrow Function

Understanding the differences is important because they behave differently, especially with **hoisting**, **this**, and modern JavaScript development.

1. Function Declaration

A function declaration is the traditional way of creating a function.

Syntax

```
JavaScript
function greet() {
  console.log("Hello");
}
```

Calling the Function

```
JavaScript
greet();
```

Output:

```
JavaScript
Hello
```

Example

```
JavaScript
function add(a, b) {
  return a + b;
}

console.log(add(10, 20));
```

Output:

```
JavaScript
30
```

Hoisting in Function Declaration

One special feature is that function declarations are fully hoisted.

```
JavaScript
```

```
sayHello();

function sayHello() {
  console.log("Hello");
}
```

Output:

```
JavaScript
Hello
```

Why?

During the memory creation phase, the entire function is stored in memory before execution begins.

2. Function Expression

A function can also be stored inside a variable.

Syntax

```
JavaScript
const greet = function() {
  console.log("Hello");
};
```

Calling

```
JavaScript
greet();
```

Output:

```
JavaScript
Hello
```

Example

```
JavaScript
const multiply = function(a, b) {
  return a * b;
};

console.log(multiply(5, 4));
```

Output:

```
JavaScript
20
```


Hoisting in Function Expression

```
JavaScript
greet();

const greet = function() {
  console.log("Hello");
};
```

Output:

```
JavaScript
ReferenceError
```

Why?

Only the variable declaration is hoisted.

```
JavaScript
const greet;
```

The function itself is assigned later during execution.

Anonymous Function

A function expression often uses an anonymous function.

```
JavaScript
const greet = function() {
  console.log("Hello");
};
```

The function has no name.

This is called an **anonymous function**.

Named Function Expression

```
JavaScript
const greet = function sayHello() {
  console.log("Hello");
};
```

Here:

```
JavaScript
sayHello
```

is the function's internal name.

3. Arrow Functions

Introduced in ES6 (2015).

Arrow functions provide shorter syntax.

Traditional Function

```
JavaScript
function add(a, b) {
  return a + b;
}
```

Arrow Function

```
JavaScript
const add = (a, b) => {
  return a + b;
};
```

Arrow Function with Single Expression

If there is only one expression, return is implicit.

```
JavaScript
const add = (a, b) => a + b;

console.log(add(10, 20));
```

Output:

```
JavaScript
30
```

Equivalent to:

```
JavaScript
const add = (a, b) => {
  return a + b;
};
```

Arrow Function with One Parameter

Parentheses are optional.

```
JavaScript
const square = num => num * num;

console.log(square(5));
```

Output:

JavaScript
25

Arrow Function with No Parameters

```
JavaScript
const greet = () => {
  console.log("Hello");
};

greet();
```

Output:

```
JavaScript
Hello
```

Comparing All Three

Function Declaration

```
JavaScript
function greet() {
  console.log("Hello");
}
```

Function Expression

```
JavaScript
const greet = function() {
  console.log("Hello");
};
```

Arrow Function

```
JavaScript
const greet = () => {
  console.log("Hello");
};
```

All produce the same output.

Hoisting Comparison

Function Declaration

```
JavaScript
sayHello();

function sayHello() {
  console.log("Hello");
}
```

✓ Works

Function Expression

```
JavaScript
sayHello();

const sayHello = function() {
  console.log("Hello");
};
```

✗ Error

Arrow Function

```
JavaScript
sayHello();

const sayHello = () => {
  console.log("Hello");
};
```

✗ Error

Modern Development Usage

Today, you'll commonly see:

```
JavaScript
const getData = () => {
  console.log("Fetching data...");
};
```

Why?

- Shorter syntax
- Cleaner code
- Widely used in React
- Preferred in modern JavaScript

Quick Interview Comparison

Feature	Function Declaration	Function Expression	Arrow Function
Syntax	Traditional	Stored in variable	Short syntax
Hoisted	✓ Yes	✗ No	✗ No
Anonymous possible	✗ Usually named	✓ Yes	✓ Yes
Modern Usage	Medium	High	Very High
React Usage	Rare	Sometimes	Very Common

Visualization

FUNCTION DECLARATION

```
function add(a,b){
  return a+b;
}
```

↓

Stored directly in memory
Hoisted completely

FUNCTION EXPRESSION

```
const add = function(a,b){
  return a+b;
}
```

↓

Variable hoisted
Function assigned later

ARROW FUNCTION

```
const add = (a,b) => a+b;
```

↓

Short syntax
Variable hoisted
Function assigned later

Exam/Interview Questions

What is a Function Declaration?

A function created using the `function` keyword with a name.

What is a Function Expression?

A function stored inside a variable.

What is an Arrow Function?

A shorter ES6 syntax for writing functions using the `=>` operator.

Which functions are hoisted?

Only **Function Declarations** are fully hoisted.

Which type is most used in React?

Arrow Functions.

Next topic: **Scope (Global Scope, Function Scope, Block Scope)**. This is where you'll learn why some variables are accessible everywhere while others exist only inside a function or block.

Topic 3: Parameters and Arguments

Topic 3: Parameters and Arguments

Parameters and arguments allow functions to receive data and work dynamically instead of using hardcoded values.

Why Do We Need Parameters?

Without parameters:

```
JavaScript
function greet() {
  console.log("Hello Krishna");
}

greet();
```

Output:

```
Hello Krishna
```

This function can only greet Krishna.

What if we want to greet Rahul, Amit, and Priya?

Instead of creating multiple functions, we use parameters.

```
JavaScript
function greet(name) {
  console.log("Hello " + name);
}

greet("Krishna");
greet("Rahul");
greet("Priya");
```

Output:

```
Hello Krishna
Hello Rahul
Hello Priya
```

What are Parameters?

Parameters are variables declared in the function definition.

```
JavaScript
function greet(name) {
  console.log(name);
}
```

Here:

```
JavaScript
name
```

is a **parameter**.

Think of parameters as placeholders that will receive values later.

What are Arguments?

Arguments are the actual values passed to a function when it is called.

```
JavaScript
greet("Krishna");
```

Here:

```
JavaScript
"Krishna"
```

is an **argument**.

Easy Analogy

Imagine a parcel box.

```
Function = Box
Parameter = Empty space inside box
Argument = Item placed inside box
```

Example:

```
JavaScript
function greet(name) {
  console.log(name);
}
```

```
greet("Krishna");
```

Parameter → name
Argument → "Krishna"

Multiple Parameters

A function can accept multiple parameters.

```
JavaScript
function add(a, b) {
  console.log(a + b);
}

add(10, 20);
```

Output:

```
30
```

Mapping:

```
a → 10
b → 20
```

Example with Three Parameters

```
JavaScript
function introduce(name, age, city) {
  console.log(name);
  console.log(age);
  console.log(city);
}

introduce("Krishna", 20, "Kanpur");
```

Output:

```
Krishna
20
Kanpur
```

Parameters vs Arguments

Parameters	Arguments
Declared in function definition	Passed during function call
Act as placeholders	Actual values
Created when function is defined	Supplied when function is executed

Example:


```
JavaScript
function add(x, y) {
  return x + y;
}

add(5, 10);
```

Parameters → x, y
Arguments → 5, 10

What Happens Internally?

```
JavaScript
function add(a, b) {
  return a + b;
}

add(5, 10);
```

When called:

```
a = 5
b = 10
```

JavaScript creates a local memory for the function.

```
Function Memory

a → 5
b → 10
```

Then:

```
JavaScript
return a + b;
```

becomes:

```
JavaScript
return 5 + 10;
```

Result:

```
15
```

Missing Arguments

What if fewer arguments are passed?

```
JavaScript
function add(a, b) {
  console.log(a);
  console.log(b);
}
```

```
add(10);
```

Output:

```
10
undefined
```

Why?

```
a → 10
b → undefined
```

JavaScript automatically assigns `undefined` to missing arguments.

Extra Arguments

What if more arguments are passed?

```
JavaScript
function add(a, b) {
  console.log(a + b);
}

add(10, 20, 30, 40);
```

Output:

```
30
```

The extra arguments are ignored by normal parameters.

Default Parameters (ES6)

We can provide default values.

```
JavaScript
function greet(name = "Guest") {
  console.log("Hello " + name);
}

greet();
```

Output:

```
Hello Guest
```

Calling with an argument:

```
JavaScript
greet("Krishna");
```

Output:

```
Hello Krishna
```

Parameter Order Matters

```
JavaScript
function introduce(name, age) {
  console.log(name, age);
}

introduce("Krishna", 20);
```

Output:

```
Krishna 20
```

If order changes:

```
JavaScript
introduce(20, "Krishna");
```

Output:

```
20 Krishna
```

Arguments are assigned by position, not by name.

Real-World Example

Calculate Total Price

```
JavaScript
function calculateTotal(price, quantity) {
  return price * quantity;
}

let total = calculateTotal(500, 3);
console.log(total);
```

Output:

```
1500
```

Mapping:

```
price    → 500
quantity → 3
```

Interview Questions

What is a parameter?

A parameter is a variable declared in the function definition that receives values when the function is called.

What is an argument?

An argument is the actual value passed to a function during function invocation.

What happens if an argument is missing?

JavaScript assigns `undefined` to the corresponding parameter.

Example:

```
JavaScript
function greet(name) {
  console.log(name);
}

greet();
```

Output:

```
undefined
```

What are default parameters?

Default parameters allow assigning a default value to a parameter if no argument is provided.

```
JavaScript
function greet(name = "Guest") {
  console.log(name);
}
```

Memory Visualization

```
JavaScript
function multiply(a, b) {
  return a * b;
}

multiply(4, 5);
```

During execution:

```
Global Memory
multiply → function
```

Function Execution Context:

Local Memory

a → 4
b → 5

Calculation:

4 × 5 = 20

Return:

20

Key Takeaway

- **Parameters** are placeholders in the function definition.
- **Arguments** are actual values passed during the function call.
- Missing arguments become **undefined**.
- Extra arguments are ignored by normal parameters.
- ES6 introduced **default parameters** to handle missing values gracefully.

Simple Formula to Remember

Function Definition → Parameters
Function Call → Arguments

Example:

```
JavaScript
function add(a, b) { // Parameters
  return a + b;
}

add(10, 20); // Arguments
```

Next topic: **Return Keyword and Function Output** — where you'll learn how functions send values back and how **return** affects execution flow.

Topic 4: Return Values

Topic 4: Return Values

A **return value** is the value that a function sends back to the place where it was called.

The **return** keyword is used to send data out of a function.

Why Do We Need Return Values?

Consider this function:

```
JavaScript
function add(a, b) {
  console.log(a + b);
}

add(10, 20);
```

Output:

```
30
```

The result is displayed, but we cannot store or reuse it.

What if we want to use the result later?

```
JavaScript
function add(a, b) {
  return a + b;
}

let result = add(10, 20);

console.log(result);
```

Output:

```
30
```

Now the result can be stored and reused.

Syntax of Return

```
JavaScript
function functionName() {
  return value;
}
```

Example:

```
JavaScript
function greet() {
  return "Hello World";
}

let message = greet();

console.log(message);
```

Output:

```
Hello World
```

How Return Works

```
JavaScript
function add(a, b) {
  return a + b;
}

let answer = add(5, 10);
```

Execution Flow

1. Function is called.
2. `a = 5, b = 10`
3. `a + b = 15`
4. `return 15`
5. Function ends.
6. `answer` receives 15.

Memory View:

```
Global Memory
answer → 15
```

Return vs Console.log()

Many beginners confuse these.

Using console.log()

```
JavaScript
function add(a, b) {
  console.log(a + b);
}

let result = add(10, 20);
console.log(result);
```

Output:

```
30
undefined
```

Why?

Because `console.log()` only prints the value.

The function does not return anything.

Using return

```
JavaScript
function add(a, b) {
  return a + b;
}

let result = add(10, 20);

console.log(result);
```

Output:

```
30
```

Because the function actually sends the value back.

Return Immediately Stops Execution

```
JavaScript
function test() {
  console.log("First");

  return "Done";

  console.log("Second");
}

test();
```

Output:

```
First
```

The code after `return` never executes.

Unreachable Code

```
JavaScript
function demo() {
  return "Hello";

  console.log("This will never run");
}
```

Everything after `return` is called **unreachable code**.

JavaScript ignores it.

Returning Different Data Types

Functions can return any data type.

Returning a Number

```
JavaScript
function square(num) {
  return num * num;
}

console.log(square(5));
```

Output:

```
25
```

Returning a String

```
JavaScript
function greet(name) {
  return "Hello " + name;
}

console.log(greet("Krishna"));
```

Output:

```
Hello Krishna
```

Returning a Boolean

```
JavaScript
function isAdult(age) {
  return age >= 18;
}

console.log(isAdult(20));
```

Output:

```
true
```

Returning an Array

```
JavaScript
function getColors() {
  return ["Red", "Blue", "Green"];
}

console.log(getColors());
```

Output:

```
["Red", "Blue", "Green"]
```

Returning an Object

```
JavaScript
function getUser() {
  return {
    name: "Krishna",
    age: 20
  };
}

console.log(getUser());
```

Output:

```
JavaScript
{
  name: "Krishna",
  age: 20
}
```

Returning the Result of Another Function

```
JavaScript
function add(a, b) {
  return a + b;
}

function squareSum(x, y) {
  return add(x, y) ** 2;
}

console.log(squareSum(2, 3));
```

Output:

```
25
```

Explanation:

```
add(2,3) = 5
```

```
52 = 25
```

Storing Return Values

```
JavaScript
function multiply(a, b) {
  return a * b;
}

let result = multiply(4, 5);

console.log(result);
```

Output:

```
20
```

Using Return Directly

```
JavaScript
function multiply(a, b) {
  return a * b;
}

console.log(multiply(4, 5));
```

Output:

```
20
```

What Happens if No Return is Written?

```
JavaScript
function greet() {
  console.log("Hello");
}

let result = greet();

console.log(result);
```

Output:

```
Hello
undefined
```

JavaScript automatically returns:

```
JavaScript
undefined
```

Explicitly Returning Nothing

```
JavaScript
function demo() {
  return;
}

console.log(demo());
```

Output:

```
undefined
```

Real-World Example

Calculate Discounted Price

```
JavaScript
function getFinalPrice(price, discount) {
  return price - discount;
}

let finalPrice = getFinalPrice(1000, 200);

console.log(finalPrice);
```

Output:

```
800
```

Memory Visualization

```
JavaScript
function add(a, b) {
  return a + b;
}

let result = add(10, 20);
```

Global Memory

```
add → function
result → ?
```

Function Execution Context

```
a → 10
b → 20
```

Calculation:

```
10 + 20 = 30
```

Return:

```
result → 30
```

Final Memory:

```
add → function
result → 30
```

Interview Questions

What is a return value?

A return value is the value sent back from a function using the **return** keyword.

What is the difference between `return` and `console.log()`?

<code>return</code>	<code>console.log()</code>
Sends value back	Displays value on screen
Can be stored in variables	Cannot be stored automatically
Ends function execution	Does not end function

What happens after a `return` statement executes?

The function immediately stops execution and exits.

What does a function return if no `return` statement exists?

JavaScript automatically returns `undefined`.

Quick Summary

```
JavaScript
function add(a, b) {
  return a + b;
}

let result = add(5, 10);
```

```
a = 5
b = 10

5 + 10 = 15

return 15

result = 15
```

Remember

```
console.log() → Show value
return → Give value back
```

A function becomes truly useful when it **returns** a value, because that value can be stored, reused, passed to another function, or displayed later.

Next Topic: Scope (Global Scope, Function Scope, and Block Scope) — one of the most important concepts in JavaScript and a foundation for understanding Execution Context and Closures.

Topic 5: Default and Rest Parameters

Topic 5: Default Parameters and Rest Parameters

ES6 introduced two very useful features for function parameters:

1. **Default Parameters** → Provide default values when arguments are missing.
2. **Rest Parameters** → Collect multiple arguments into a single array.

1. Default Parameters

Default parameters allow us to assign a default value to a parameter if the caller doesn't provide one.

Problem Before ES6

```
JavaScript
function greet(name) {
  console.log("Hello " + name);
}

greet();
```

Output:

```
Hello undefined
```

Because no argument was passed.

Solution: Default Parameter

```
JavaScript
function greet(name = "Guest") {
  console.log("Hello " + name);
}

greet();
```

Output:

```
Hello Guest
```

Now JavaScript uses "Guest" whenever no value is provided.

When Argument is Provided

```
JavaScript
function greet(name = "Guest") {
  console.log("Hello " + name);
}
```

```
greet("Krishna");
```

Output:

```
Hello Krishna
```

The passed argument overrides the default value.

Multiple Default Parameters

```
JavaScript
function introduce(name = "Unknown", age = 0) {
    console.log(name, age);
}

introduce();
```

Output:

```
Unknown 0
```

Partial Arguments

```
JavaScript
function introduce(name = "Unknown", age = 0) {
    console.log(name, age);
}

introduce("Krishna");
```

Output:

```
Krishna 0
```

Only the missing parameter gets the default value.

Default Parameter Expressions

Default values can also be expressions.

```
JavaScript
function calculate(price, tax = price * 0.18) {
    return price + tax;
}

console.log(calculate(1000));
```

Output:

Real-World Example

```
JavaScript
function createUser(username, role = "user") {
  return `${username} is a ${role}`;
}

console.log(createUser("Krishna"));
console.log(createUser("Admin", "administrator"));
```

Output:

```
Krishna is a user
Admin is a administrator
```

2. Rest Parameters

Sometimes we don't know how many arguments a function will receive.

Example:

```
JavaScript
add(10, 20);
add(10, 20, 30);
add(10, 20, 30, 40);
```

How do we handle all of them?

That's where **Rest Parameters** come in.

Syntax

```
JavaScript
function functionName(...parameterName) {
  // code
}
```

The three dots (...) are called the **rest operator**.

Example

```
JavaScript
function collectNumbers(...numbers) {
  console.log(numbers);
}

collectNumbers(10, 20, 30, 40);
```


Output:

```
JavaScript
[10, 20, 30, 40]
```

JavaScript gathers all arguments into an array.

Sum Any Number of Values

```
JavaScript
function sum(...numbers) {
  let total = 0;

  for (let num of numbers) {
    total += num;
  }

  return total;
}

console.log(sum(10, 20));
console.log(sum(10, 20, 30));
console.log(sum(10, 20, 30, 40));
```

Output:

```
30
60
100
```

Rest Parameter is an Array

```
JavaScript
function demo(...items) {
  console.log(Array.isArray(items));
}

demo(1, 2, 3);
```

Output:

```
true
```

Therefore array methods can be used.

```
JavaScript
function demo(...items) {
  console.log(items.length);
}

demo(10, 20, 30, 40);
```

Output:

```
4
```

Combining Normal and Rest Parameters

```
JavaScript
function introduce(name, ...hobbies) {
  console.log(name);
  console.log(hobbies);
}

introduce("Krishna", "Coding", "Reading", "Gaming");
```

Output:

```
Krishna
["Coding", "Reading", "Gaming"]
```

Mapping:

```
name → "Krishna"

hobbies → [
  "Coding",
  "Reading",
  "Gaming"
]
```

Important Rule

The rest parameter must always be the last parameter.

✅ Correct:

```
JavaScript
function demo(a, b, ...rest) {
}
```

❌ Wrong:

```
JavaScript
function demo(...rest, a) {
}
```

This causes a syntax error.

Default Parameters + Rest Parameters

```
JavaScript
function order(item = "Burger", ...extras) {
  console.log(item);
  console.log(extras);
}

order("Pizza", "Cheese", "Sauce");
```

Output:

```
Pizza
["Cheese", "Sauce"]
```

Rest Parameters vs Arguments Object

Before ES6, JavaScript provided an `arguments` object.

```
JavaScript
function demo() {
  console.log(arguments);
}

demo(10, 20, 30);
```

Output:

```
JavaScript
Arguments(3) [10, 20, 30]
```

Problems with arguments

- Not a real array
- Cannot directly use array methods
- Not available in arrow functions

Modern Solution

```
JavaScript
function demo(...numbers) {
  console.log(numbers);
}
```

Output:

```
JavaScript
[10, 20, 30]
```

Rest parameters are cleaner and easier to use.

Real-World Example

Shopping Cart

```
JavaScript
function cartTotal(...prices) {
  return prices.reduce((sum, price) => sum + price, 0);
}
```

```
console.log(cartTotal(500, 1000, 200, 300));
```

Output:

```
2000
```

The function can handle any number of products.

Memory Visualization

```
JavaScript
function sum(...numbers) {
  return numbers;
}

sum(10, 20, 30, 40);
```

Function Memory:

```
numbers →
[
  10,
  20,
  30,
  40
]
```

Instead of creating multiple variables, JavaScript stores everything in one array.

Interview Questions

What are default parameters?

Default parameters provide fallback values when arguments are not supplied.

Example:

```
JavaScript
function greet(name = "Guest") {
  return name;
}
```

What are rest parameters?

Rest parameters collect multiple arguments into a single array.

Example:

```
JavaScript
function sum(...numbers) {
  return numbers;
}
```

What symbol is used for rest parameters?

JavaScript

...

(three dots)

Can rest parameters be placed anywhere?

No.

They must be the last parameter.

Difference Between Default and Rest Parameters

Default Parameter	Rest Parameter
Provides fallback value	Collects multiple values
Used when arguments are missing	Used when argument count is unknown
One parameter gets one value	One parameter gets many values
Example: <code>name = "Guest"</code>	Example: <code>...numbers</code>

Quick Summary

Default Parameters

JavaScript

```
function greet(name = "Guest") {
  return name;
}
```

Missing argument → Uses default value

Rest Parameters

JavaScript

```
function sum(...numbers) {
  return numbers;
}
```

Many arguments → Stored in one array

Easy Formula

```
Default Parameter
=
Fallback Value

Rest Parameter
=
Collect Unlimited Values
```

Examples to Remember

```
JavaScript
function greet(name = "Guest") {
  console.log(name);
}
```

```
JavaScript
function sum(...numbers) {
  console.log(numbers);
}
```

These two ES6 features are heavily used in modern JavaScript, React, Node.js, and API development.

Next Topic: Callback Functions and Higher-Order Functions

This topic is the bridge between basic JavaScript and advanced concepts like asynchronous programming, event handling, promises, and React.

Topic 6: Execution Context - Global Execution Context, Function Execution Context

Topic 6: Execution Context

Global Execution Context (GEC) and Function Execution Context (FEC)

This is one of the most important JavaScript concepts.

Many developers can write functions, but understanding **Execution Context** helps you understand **how JavaScript actually runs your code behind the scenes**.

What is Execution Context?

An **Execution Context** is the environment in which JavaScript code is executed.

Whenever JavaScript runs code, it creates an execution context containing:

1. Memory (Variables and Functions)
2. Code Execution Area

Think of it as a workspace where JavaScript executes code.

Real-Life Analogy

Imagine a student taking an exam.

Before solving questions:

- Answer sheet is given.
- Roll number is written.
- Personal details are filled.

Then:

- Questions are solved.

Similarly, JavaScript first prepares memory and then executes code.

Types of Execution Context

There are mainly two types:

1. Global Execution Context (GEC)
2. Function Execution Context (FEC)

Global Execution Context (GEC)

When a JavaScript program starts, JavaScript creates a **Global Execution Context**.

There is only **one GEC** for a program.

Example:

```
JavaScript
let a = 10;

function greet() {
  console.log("Hello");
}

console.log(a);
greet();
```

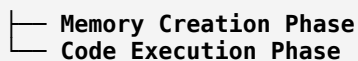
Before execution starts:

Global Execution Context Created

Structure of Global Execution Context

The GEC has two parts:

Global Execution Context



Phase 1: Memory Creation Phase

Also called:

Creation Phase

JavaScript scans the entire code.

It allocates memory for:

- Variables
- Functions

Example:

```

JavaScript
let a = 10;

function greet() {
  console.log("Hello");
}
  
```

Memory Phase:

Memory

a → undefined

greet → complete function code

Notice:

- Variables get **undefined**
- Functions store their entire definition

Phase 2: Code Execution Phase

Now JavaScript executes code line by line.

```

JavaScript
let a = 10;
  
```

Updates:

a → 10

Next:


```
JavaScript
greet();
```

Function is called.

At this moment JavaScript creates a new Execution Context.

Function Execution Context (FEC)

Whenever a function is invoked, JavaScript creates a separate Function Execution Context.

Example:

```
JavaScript
function greet() {
  console.log("Hello");
}

greet();
```

When `greet()` executes:

```
Function Execution Context Created
```

Visualization

```
Global Execution Context

|
|---- greet()
|      |
|      ↓
Function Execution Context
```

Example 1

```
JavaScript
let a = 10;

function square(num) {
  return num * num;
}

let result = square(5);
```

Step 1: Global Memory Creation

```
Global Memory

a      → undefined
square → function
```

```
result → undefined
```

Step 2: Global Code Execution

```
JavaScript
a = 10;
```

Memory:

```
a → 10
```

Next:

```
JavaScript
square(5);
```

Function called.

Function Execution Context Creation

A new context is created.

Memory Phase:

```
Function Memory
num → undefined
```

Execution Phase:

```
JavaScript
num = 5
```

Memory:

```
num → 5
```

Calculation:

```
JavaScript
return num * num;
```

Becomes:

```
JavaScript
return 25;
```

Returning to Global Context

Returned value:

```
25
```

Stored in:

```
JavaScript
result = 25;
```

Final Memory:

```
a      → 10
square → function
result → 25
```

Function context is destroyed after execution.

Important Concept

Each function call gets its own execution context.

Example:

```
JavaScript
function greet(name) {
  console.log(name);
}

greet("Krishna");
greet("Rahul");
```

JavaScript creates:

```
FEC #1
name → Krishna
```

Destroyed.

Then:

```
FEC #2
name → Rahul
```

Destroyed.

Each call gets fresh memory.

Example 2 (Nested Functions)

```
JavaScript
```

```
function one() {
  console.log("One");

  function two() {
    console.log("Two");
  }

  two();
}

one();
```

Execution:

Global Context

↓

one() Context

↓

two() Context

Each function call creates its own context.

What Does an Execution Context Contain?

Every execution context contains:

1. Memory Component
2. Code Component

Memory Component

Stores:

Variables
Functions
Parameters

Example:

```
JavaScript
function add(a, b) {
  let sum = a + b;
}
```

Memory:

a
b
sum

Code Component

Executes statements line by line.

```
JavaScript
sum = a + b;
```

runs here.

Complete Visualization

Code:

```
JavaScript
let x = 2;

function multiply(n) {
  return n * n;
}

let result = multiply(x);
```

Global Memory Phase

```
x      → undefined
multiply → function
result → undefined
```

Global Execution Phase

```
x → 2
```

Function called:

```
JavaScript
multiply(2)
```

Function Memory Phase

```
n → undefined
```

Function Execution Phase

```
n → 2
```

Calculation:

```
2 * 2 = 4
```

Return:

```
4
```

Back to GEC:

```
result → 4
```

Interview Questions

What is an Execution Context?

An execution context is the environment in which JavaScript code is evaluated and executed.

How many phases are there in an execution context?

Two phases:

1. Memory Creation Phase
2. Code Execution Phase

What is Global Execution Context?

The execution context created when a JavaScript program starts.

There is only one GEC per program.

What is Function Execution Context?

A new execution context created whenever a function is called.

What happens to a Function Execution Context after execution?

It is removed from memory after the function completes.

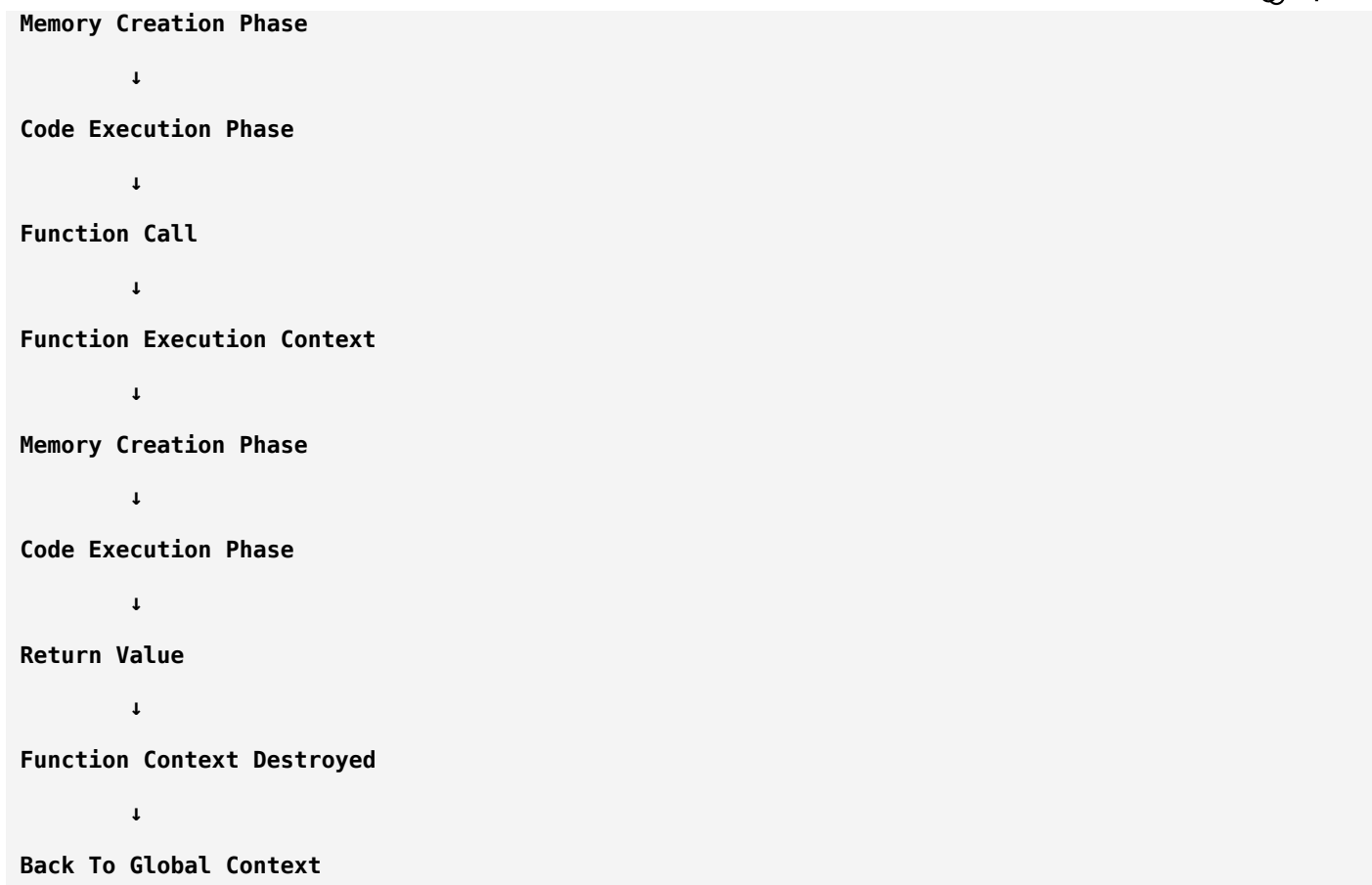
Most Important Diagram

```
JavaScript Program Starts
```

```
↓
```

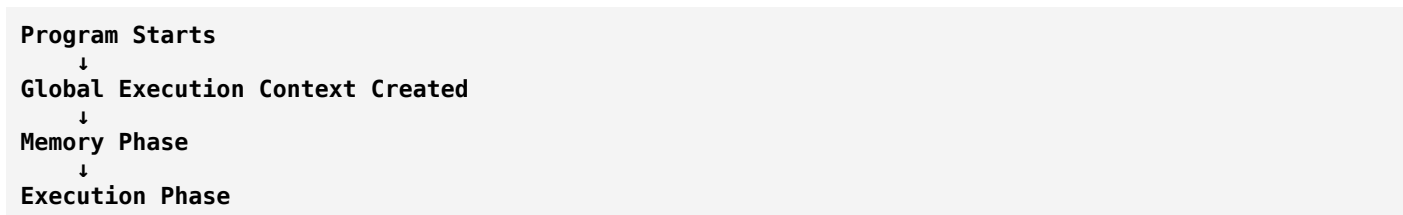
```
Global Execution Context
```

```
↓
```

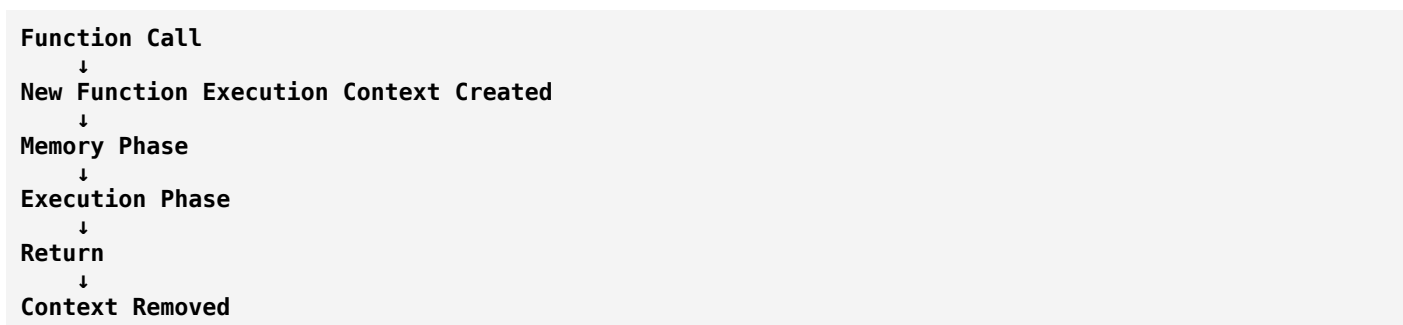


Key Takeaway

Whenever JavaScript runs code:



Whenever a function is called:



This concept is the foundation for understanding the next topic:

Call Stack (JavaScript's Execution Manager)

The Call Stack keeps track of all execution contexts and decides which function runs first and which returns next. Understanding it will make recursion, asynchronous JavaScript, promises, and React internals much easier.

Topic 7: Creation Phase and Execution Phase

Topic 7: Creation Phase and Execution Phase

In the previous topic, you learned that every **Execution Context** has two phases:

1. **Creation Phase (Memory Creation Phase)**
2. **Execution Phase (Code Execution Phase)**

These phases are the heart of how JavaScript executes code.

Big Picture

Whenever JavaScript runs a program:

```

Create Execution Context
  ↓
Creation Phase
  ↓
Execution Phase
  
```

This happens for:

- Global Execution Context (GEC)
- Function Execution Context (FEC)

Real-Life Analogy

Imagine you're preparing for an exam.

Step 1: Preparation

Before writing answers:

- Get answer sheet
- Write name
- Write roll number

This is like the **Creation Phase**.

Step 2: Solving Questions

Now you start answering questions.

This is like the **Execution Phase**.

Example Program

```
JavaScript
var x = 10;

function greet() {
  console.log("Hello");
}

var y = 20;
```

JavaScript does NOT execute line by line immediately.

First it creates memory.

Phase 1: Creation Phase

During this phase, JavaScript scans the entire code.

It allocates memory for:

- Variables
 - Functions
-

Rule for Variables

Variables get:

```
JavaScript
undefined
```

Rule for Functions

Functions get:

```
JavaScript
Entire Function Definition
```

Memory Created

For the code:

```
JavaScript
```

```
var x = 10;

function greet() {
  console.log("Hello");
}

var y = 20;
```

Memory becomes:

Memory

```
x      → undefined
greet  → function code
y      → undefined
```

No values are assigned yet.

Visualization

Creation Phase

Code Scan



```
x      → undefined
greet  → function
y      → undefined
```

Phase 2: Execution Phase

Now JavaScript starts executing code line by line.

Line 1

JavaScript

```
var x = 10;
```

Update memory:

```
x → 10
```

Function Declaration

JavaScript

```
function greet() {
  console.log("Hello");
}
```

Nothing happens.

The function is already stored in memory.

Next Line

```
JavaScript  
var y = 20;
```

Update:

```
y → 20
```

Final Memory

```
x      → 10  
greet  → function  
y      → 20
```

Complete Flow

```
Code  
var x = 10;  
function greet(){}  
var y = 20;
```

Creation Phase

```
x → undefined  
greet → function  
y → undefined
```

Execution Phase

```
x → 10  
greet → function  
y → 20
```

Example with Function Call

```
JavaScript  
var x = 10;
```

```
function square(num) {
  return num * num;
}

var result = square(5);
```

Global Creation Phase

Memory:

```
x      → undefined
square → function
result → undefined
```

Global Execution Phase

Line 1

```
JavaScript
x = 10
```

Memory:

```
x → 10
```

Function Call

```
JavaScript
square(5)
```

Now a Function Execution Context is created.

Function Creation Phase

Memory:

```
num → undefined
```

Function Execution Phase

Argument assigned:

```
num → 5
```

Calculation:

```
JavaScript
return num * num;
```

Becomes:

```
JavaScript
return 25;
```

Function returns:

```
25
```

Back to Global Context

```
JavaScript
result = 25;
```

Final Memory:

```
x      → 10
square → function
result → 25
```

What Happens with let and const?

This is where things get interesting.

Using var

```
JavaScript
console.log(a);

var a = 10;
```

Creation Phase:

```
a → undefined
```

Execution:

```
JavaScript
console.log(a);
```

Output:

```
undefined
```

No error.

Using let

```
JavaScript
console.log(a);

let a = 10;
```

Output:

```
ReferenceError
```

Why?

Memory is allocated during creation phase, but the variable cannot be accessed before initialization.

This period is called:

```
Temporal Dead Zone (TDZ)
```

Using const

```
JavaScript
console.log(a);

const a = 10;
```

Output:

```
ReferenceError
```

Same reason.

Important Interview Question

What is Hoisting?

Hoisting is JavaScript's behavior of allocating memory for variables and functions during the Creation Phase before code execution begins.

Example:

```
JavaScript
sayHello();

function sayHello() {
  console.log("Hello");
}
```

Output:

Hello

Why?

Creation Phase:

sayHello → function definition

So the function already exists before execution starts.

Function Creation Phase vs Function Execution Phase

Code:

```
JavaScript
function add(a, b) {
  return a + b;
}

add(5, 10);
```

Function Creation Phase

Memory:

```
a → undefined
b → undefined
```

Function Execution Phase

Arguments assigned:

```
a → 5
b → 10
```

Then:

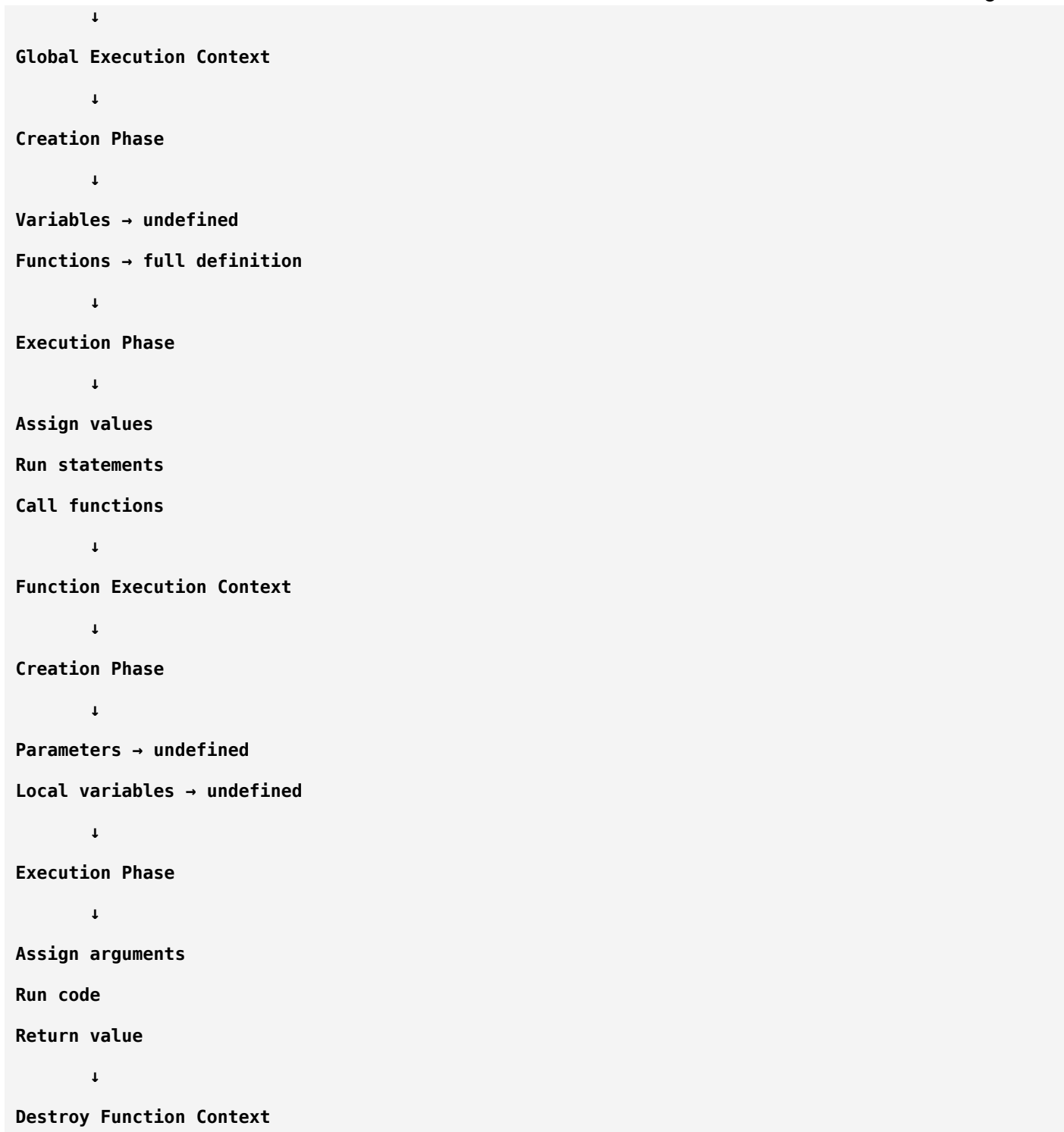
```
JavaScript
return a + b;
```

Result:

15

Visualization of Entire Process

Program Starts



Interview Questions

What happens in the Creation Phase?

JavaScript allocates memory for variables and functions.

- Variables → **undefined**
- Functions → complete function definition

What happens in the Execution Phase?

JavaScript executes code line by line and assigns actual values.

Which phase causes hoisting?

The Creation Phase.

Why are functions callable before their declaration?

Because their entire definition is stored during the Creation Phase.

What value do var variables get during Creation Phase?

```
JavaScript  
undefined
```

What value do function declarations get during Creation Phase?

Their complete function definition.

Quick Summary

Creation Phase

```
Allocate Memory  
var → undefined  
function → complete definition
```

Execution Phase

```
Assign Values  
Run Code  
Call Functions  
Return Results
```

Easy Formula

```
Creation Phase  
=  
Memory Allocation  
  
Execution Phase  
=  
Code Execution
```

Most Important Example

```
JavaScript
console.log(x);

var x = 10;
```

Creation Phase

```
x → undefined
```

Execution Phase

```
JavaScript
console.log(x);
```

Output:

```
undefined
```

Then:

```
JavaScript
x = 10;
```

Memory:

```
x → 10
```

This example perfectly demonstrates the difference between the **Creation Phase** and the **Execution Phase**.

Next Topic: Hoisting (Variables and Functions)

You'll see how the Creation Phase leads directly to hoisting and why **var**, **let**, **const**, function declarations, and function expressions behave differently.

Topic 8: Hoisting

Topic 8: Hoisting

Hoisting is one of the most frequently asked JavaScript interview topics.

Many beginners think JavaScript physically moves variables and functions to the top of the file. That's **not true**.

Hoisting is a result of the **Creation Phase** of the Execution Context.

What is Hoisting?

Definition:

Hoisting is JavaScript's behavior of allocating memory for variables and functions before code execution begins.

In simple words:

```
JavaScript first scans the code,
creates memory for variables/functions,
and then executes the code.
```

Because of this, some variables and functions can be accessed before they appear in the code.

Example 1: var Hoisting

```
JavaScript
console.log(a);
var a = 10;
```

Output:

```
undefined
```

Many beginners expect:

```
ReferenceError
```

But JavaScript outputs:

```
undefined
```

What Happens Internally?

Creation Phase

JavaScript scans the code.

Memory:

```
a → undefined
```

Execution Phase

```
JavaScript
console.log(a);
```

Current value:

```
undefined
```

Output:

```
undefined
```

Then:

```
JavaScript
a = 10;
```

Memory becomes:

```
a → 10
```

Visualization

```
Code
console.log(a);
var a = 10;
```

Creation Phase

```
a → undefined
```

Execution Phase

```
console.log(a)
```

Output:

```
undefined
```

Then:

```
a → 10
```

Example 2: Function Hoisting

```
JavaScript
sayHello();

function sayHello() {
  console.log("Hello");
}
```

Output:

```
Hello
```

Why Does It Work?

Creation Phase

Memory:

```
sayHello → complete function definition
```

Execution Phase

```
JavaScript
sayHello();
```

Function already exists in memory.

Output:

```
Hello
```

Function Declaration Hoisting

Function declarations are **fully hoisted**.

```
JavaScript
greet();

function greet() {
  console.log("Hi");
}
```

✓ Works

Because the complete function is stored during the Creation Phase.

Function Expression Hoisting

```
JavaScript
greet();

var greet = function() {
  console.log("Hi");
};
```

Output:

```
TypeError: greet is not a function
```

Why?

Creation Phase

```
greet → undefined
```

Only the variable is hoisted.

The function assignment happens later.

Execution Phase

```
JavaScript
greet();
```

Currently:

```
greet = undefined
```

So:

```
JavaScript
undefined();
```

Which causes:

```
TypeError
```

Arrow Function Hoisting

```
JavaScript
greet();

const greet = () => {
  console.log("Hello");
};
```

Output:

```
ReferenceError
```

Why?

Because `const` variables are in the **Temporal Dead Zone (TDZ)** until initialized.

Hoisting with let

```
JavaScript
console.log(a);

let a = 10;
```

Output:

```
ReferenceError
```

Hoisting with const

```
JavaScript
console.log(a);

const a = 10;
```

Output:

```
ReferenceError
```

Important Truth

Many people say:

```
let and const are not hoisted
```

This is technically incorrect.

They **are hoisted**, but they are placed in a special area called the **Temporal Dead Zone (TDZ)**.

Temporal Dead Zone (TDZ)

The TDZ is the time between:

```
Memory Allocation
↓
Variable Initialization
```

During this period, accessing the variable causes an error.

Example:

```
JavaScript
console.log(age);

let age = 20;
```

Visualization:

```
Creation Phase
age → allocated memory
```

But:

```
age is in TDZ
```

Execution:

```
JavaScript
```

```
console.log(age);
```

Output:

```
ReferenceError
```

TDZ Timeline

```
JavaScript
console.log(name);
let name = "Krishna";
```

```
Program Starts
  ↓
Memory Allocated
  ↓
TDZ Starts
  ↓
console.log(name)
  ↓
ReferenceError
  ↓
name = "Krishna"
  ↓
TDZ Ends
```

Hoisting Comparison Table

Type	Hoisted?	Initial Value
var	✓ Yes	undefined
let	✓ Yes	TDZ
const	✓ Yes	TDZ
Function Declaration	✓ Yes	Full Function
Function Expression (var)	Partially	undefined
Arrow Function (const)	Partially	TDZ

Example: var vs let

```
JavaScript
console.log(a);
var a = 10;
```

Output:

```
undefined
```

```
JavaScript
console.log(b);
let b = 10;
```


Output:

ReferenceError

Example: Function Declaration

```
JavaScript
add();

function add() {
  console.log("Added");
}
```

Output:

Added

Example: Function Expression

```
JavaScript
add();

var add = function() {
  console.log("Added");
};
```

Output:

TypeError

Example: Arrow Function

```
JavaScript
add();

const add = () => {
  console.log("Added");
};
```

Output:

ReferenceError

Why Was let/const Introduced?

var caused many bugs because:

```
JavaScript
console.log(a);

var a = 10;
```

returns:

```
undefined
```

which can be confusing.

ES6 introduced:

```
JavaScript
let
const
```

to make variable usage safer.

Memory Visualization

Code:

```
JavaScript
console.log(a);

var a = 10;

function greet() {
  console.log("Hello");
}
```

Creation Phase

```
Memory

a      → undefined
greet  → function definition
```

Execution Phase

```
JavaScript
console.log(a);
```

Output:

```
undefined
```

Then:

```
JavaScript
a = 10;
```

Memory:

```
a → 10
```

Interview Questions

What is Hoisting?

Hoisting is JavaScript's behavior of allocating memory for variables and functions before code execution starts.

Which variables are hoisted?

```
var
let
const
```

All are hoisted.

Why does `var` give undefined?

Because during the Creation Phase:

```
var → undefined
```

Why do `let` and `const` give `ReferenceError`?

Because they remain in the Temporal Dead Zone until initialization.

Are function declarations hoisted?

Yes.

The entire function definition is hoisted.

Are function expressions fully hoisted?

No.

Only the variable declaration is hoisted.

What is the Temporal Dead Zone?

The period between memory allocation and initialization of a `let` or `const` variable.

Most Important Interview Diagram

Creation Phase

```
var x      → undefined
```

```
let y      → TDZ
```

```
const z    → TDZ
greet()    → full function
```

Execution Phase

```
x gets value
y initialized
z initialized
function calls execute
```

Quick Summary

var

```
JavaScript
console.log(a);
var a = 10;
```

Output:

```
undefined
```

let

```
JavaScript
console.log(a);
let a = 10;
```

Output:

```
ReferenceError
```

const

```
JavaScript
console.log(a);
const a = 10;
```

Output:

```
ReferenceError
```

Function Declaration

```
JavaScript
greet();

function greet() {}
```

Output:

Works

Function Expression

```
JavaScript
greet();

var greet = function() {};
```

Output:

TypeError

Easy Formula

```
Hoisting
=
Memory Allocation Before Execution
```

```
var
→ undefined

let/const
→ TDZ

function declaration
→ full function
```

Module 4 Status

You now understand:

- ✓ What is a Function
- ✓ Function Declaration, Expression, Arrow Functions
- ✓ Parameters and Arguments
- ✓ Return Values
- ✓ Default and Rest Parameters
- ✓ Execution Context
- ✓ Creation Phase and Execution Phase
- ✓ Hoisting

The next major topic is **Call Stack**, which shows how JavaScript manages multiple execution contexts and function calls. It is the final piece needed to fully understand how JavaScript executes code behind the scenes.

Topic 9: Scope - Global, Function, Block Scope

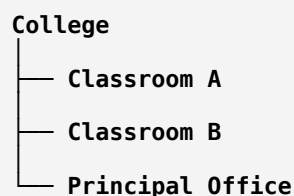
Topic 9: Scope – Global Scope, Function Scope, and Block Scope

Scope determines **where a variable can be accessed in your program**.

Think of scope as the **visibility area** of a variable.

Real-Life Analogy

Imagine a college.



- The principal can access the whole college.
- Students in Classroom A cannot directly access Classroom B.
- Each room has its own area.

Similarly, variables have their own visibility areas.

What is Scope?

Definition:

Scope is the region of a program where a variable is accessible.

Example:

```

JavaScript
let name = "Krishna";

console.log(name);
  
```

Output:

```
Krishna
```

Because **name** is inside the accessible scope.

Types of Scope in JavaScript

There are three main types:

1. Global Scope
2. Function Scope
3. Block Scope

1. Global Scope

A variable declared outside all functions and blocks belongs to the **global scope**.

Example:

```
JavaScript
let college = "ABC University";

function showCollege() {
  console.log(college);
}

showCollege();

console.log(college);
```

Output:

```
ABC University
ABC University
```

Why?

Because:

```
JavaScript
let college = "ABC University";
```

is declared globally.

Global variables are accessible everywhere in the program (unless shadowed).

Visualization

```
Global Scope

college = "ABC University"

↓
Accessible Everywhere
```

Example

```
JavaScript
let a = 10;

function test() {
  console.log(a);
}

test();
```

Output:

```
10
```

The function can access global variables.

2. Function Scope

Variables declared inside a function exist only within that function.

Example:

```
JavaScript
function greet() {
  let message = "Hello";
  console.log(message);
}

greet();
```

Output:

```
Hello
```

Accessing Outside the Function

```
JavaScript
function greet() {
  let message = "Hello";
}

console.log(message);
```

Output:

```
ReferenceError: message is not defined
```

Why?

Because `message` belongs to the function's scope.

Visualization

```
Global Scope
  ↓
greet()
message = "Hello"
```

`message` lives only inside `greet()`.

Example


```
JavaScript
function add() {
  let a = 10;
  let b = 20;

  console.log(a + b);
}

add();
```

Output:

```
30
```

Outside the function:

```
JavaScript
console.log(a);
```

Output:

```
ReferenceError
```

Function Scope with var

Historically, `var` is function-scoped.

```
JavaScript
function demo() {
  var age = 20;

  console.log(age);
}

demo();
```

Output:

```
20
```

Outside:

```
JavaScript
console.log(age);
```

Output:

```
ReferenceError
```

3. Block Scope

A block is anything inside curly braces:

```
JavaScript
{
  // block
}
```

Examples:

```
JavaScript
if () {}
for () {}
while () {}
{
}
```

let and const are Block Scoped

```
JavaScript
{
  let name = "Krishna";
  console.log(name);
}
```

Output:

```
Krishna
```

Outside:

```
JavaScript
console.log(name);
```

Output:

```
ReferenceError
```

Another Example

```
JavaScript
if (true) {
  let age = 20;
  console.log(age);
}
```

Output:

```
20
```

Outside:

```
JavaScript
console.log(age);
```

Output:

```
ReferenceError
```

const is Also Block Scoped

```
JavaScript
{
  const PI = 3.14;
  console.log(PI);
}
```

Output:

```
3.14
```

Outside:

```
JavaScript
console.log(PI);
```

Output:

```
ReferenceError
```

var is NOT Block Scoped

This is a very important interview question.

```
JavaScript
{
  var city = "Kanpur";
}

console.log(city);
```

Output:

```
Kanpur
```

Why?

Because `var` ignores block scope.

It only respects function scope.

Visualization

let

```
JavaScript
{
  let x = 10;
}
```

Block Scope

```
x = 10
```

Outside:

Not Accessible

var

```
JavaScript
{
  var x = 10;
}
```

Global Scope

```
x = 10
```

Outside:

Accessible

Scope Chain

One of the most important JavaScript concepts.

Example

```
JavaScript
let college = "ABC";

function student() {
  let name = "Krishna";
  console.log(name);
  console.log(college);
}

student();
```

Output:

```
Krishna
ABC
```

How JavaScript Searches

When JavaScript needs a variable:

```
JavaScript
console.log(college);
```

It checks:

```
Current Scope
  ↓
Parent Scope
  ↓
Global Scope
```

This lookup process is called the **Scope Chain**.

Example of Scope Chain

```
JavaScript
let a = 10;

function outer() {
  let b = 20;
  function inner() {
    let c = 30;
    console.log(a);
    console.log(b);
    console.log(c);
  }
  inner();
}

outer();
```

Output:

```
10
20
30
```

Visualization

```
Global Scope
a = 10
  ↓
outer()
b = 20
```

```

↓
inner()
c = 30

```

`inner()` can access:

```

c
b
a

```

through the scope chain.

Variable Shadowing

A local variable can hide a global variable.

Example:

```

JavaScript
let name = "Global";

function test() {
  let name = "Local";

  console.log(name);
}

test();

```

Output:

```

Local

```

The local variable shadows the global one.

Visualization

```

Global Scope
name = "Global"
↓
Function Scope
name = "Local"

```

JavaScript uses the nearest variable first.

Scope Summary Table

Scope Type	Accessible Where?
Global Scope	Entire Program
Function Scope	Inside Function Only
Block Scope	Inside Block Only

var vs let vs const

Feature	var	let	const
Global Scope	✓	✓	✓
Function Scope	✓	✓	✓
Block Scope	✗	✓	✓
Redeclaration	✓	✗	✗

Interview Questions

What is Scope?

Scope determines where a variable can be accessed.

What is Global Scope?

Variables declared outside all functions and blocks.

What is Function Scope?

Variables declared inside a function.

What is Block Scope?

Variables declared with `let` or `const` inside a block `{}`.

Is var block-scoped?

No.

`var` is function-scoped.

Are let and const block-scoped?

Yes.

Both respect block boundaries.

What is the Scope Chain?

The mechanism JavaScript uses to search for variables from the current scope outward to parent and global scopes.

Memory Visualization

```
JavaScript
let a = 10;

function demo() {
  let b = 20;

  console.log(a);
  console.log(b);
}

demo();
```

Global Memory

```
a → 10
demo → function
```

Function Memory

```
b → 20
```

When searching for a:

```
Function Scope
  ↓
Global Scope
```

Found:

```
a → 10
```

Quick Summary

```
Global Scope
=
Accessible Everywhere
```

```
Function Scope
=
Accessible Only Inside Function
```

```
Block Scope
=
```


Accessible Only Inside {}

let and const
=
Block Scoped

var
=
Function Scoped

Easy Formula

Current Scope
↓
Parent Scope
↓
Global Scope

This lookup process is called the **Scope Chain**.

Module 4 Progress

You now understand:

- ✓ Functions
- ✓ Function Declarations, Expressions, Arrow Functions
- ✓ Parameters and Arguments
- ✓ Return Values
- ✓ Default and Rest Parameters
- ✓ Execution Context
- ✓ Creation Phase and Execution Phase
- ✓ Hoisting
- ✓ Scope

Next Topic: Closures (Introduction)

Closures combine **Functions + Scope + Execution Context** and are one of the most powerful concepts in JavaScript, heavily used in React, Node.js, event handlers, and interview questions.